



University of Engineering & Technology Peshawar

Computer Programming (C++) Lab

Lab 12

STUDENT NAME	MUHAMMAD YASEEN
REGISTRATION NO:	BF25NWELE0670
COUSRE:	EE-170L COMPUTER PROGRAMMING LAB
DATE:	14/05/26
Lab Title:	POINTERS IN C++

SUBMITTED TO ENGR MUHAMMAD RIZWAN

1 Objective:

The objective of this lab session is to introduce and apply the following concepts in C++:

- Computer Memory and Variables
- Pointers
- Pointer Declaration
- Referencing Operator (&)
- Dereferencing Operator (*)

2 Computer Memory:

The computer's memory is made up of bytes. Each byte has a unique address associated with it. When a program runs, variables are stored in memory at specific addresses determined by the compiler. The size of the memory block reserved depends on the data type: for example, on a 32-bit PC an integer occupies 4 bytes, while on older 16-bit PCs integers occupied 2 bytes.

2.1 Variables and Memory:

When a variable is declared, the compiler reserves a specific block of memory to hold its value. Two distinct steps occur when a variable is accessed:

- The program locates the address reserved for the variable.
- The contents stored at that address are retrieved.

```
#include <iostream> // for input/output operations
using namespace std;
int main() {
    // Declare a variable of type float and initialize it
    float fl = 3.14;
    // Display the value of the variable
    cout << fl;
    return 0;
}
```

Output

3.14

Note: The float value 3.14 is actually stored as 32 ones and zeros (4 bytes × 8 bits) in memory, not as individual characters.

3 Pointers in C++:

A pointer is a variable that holds the memory address of another variable. Unlike regular variables that store values directly, a pointer stores the address where a value is kept. Because a pointer holds an address rather than a value, it has two components:

- The pointer variable itself (holds the address).
- The value stored at that address.

3.1 Pointer Declaration:

The syntax for declaring a pointer variable is:

```
// Syntax: dataType *variableName;
```

```
// Declare a pointer to an integer
int *ptr;
// Declare a pointer to a float
float *fptr;
// Declare a pointer to a character
char *cptr;
```

The asterisk (*) tells the compiler that a pointer variable is being created. The data type specifies the type of data at the address the pointer will hold.

3.2 Referencing Operator (&):

The address-of operator (&) retrieves the memory address of a variable and assigns it to a pointer:

```
// Declare a pointer variable of type int
int *ptr;
// Declare an integer variable
int k;
// ptr now holds the address of k using the & operator
ptr = &k;
```

After this statement, ptr is said to "point to" k.

3.3 Dereferencing Operator (*):

The dereference operator (*) accesses the value at the address stored in a pointer:

```
// Set the value of the variable ptr points to, to 7
*ptr = 7;
// Print the value stored at the address ptr points to
cout << *ptr;
```

When * is used in this context, we are referring to the value that ptr points to, not the pointer's own address value.

3.4 Example: Pointer Operations:

```
#include <iostream> // for input/output operations
using namespace std;
int main() {
    // Declare integer variables
    int a = 5;
    int b = 15;
    // Declare two pointers of type int
    int *p1, *p2;
    // Initialize p1 to point to a
    p1 = &a;
    // Initialize p2 to point to b
    p2 = &b;
    // Set the value of a to 10 using pointer p1
    *p1 = 10;
    // Set the value of b to the value pointed by p1 (which is 10)
    *p2 = *p1;
    // Set the value of a to 20 using pointer p1
    *p1 = 20;
```

```
// Display the final value of a
cout << "a = " << a << endl;

// Display the final value of b
cout << "b = " << b << endl;
return 0;
}
```

Output

```
a = 20
b = 10
```

4 Lab Tasks — Lab 12:

Task 1: Pointer Declaration and Initialization:

Declare an integer variable num, a pointer ptr, initialize the pointer to point to num, and display the values using the dereference operator.

```
#include <iostream> // for input/output operations
using namespace std;
int main() {
    // Declare an integer variable and initialize it
    int num = 42;
    // Declare a pointer variable of type int
    int *ptr;
    // Initialize pointer to point to address of num
    ptr = &num;
    // Display the value of num
    cout << "Value of num: " << num << endl;
    // Display the address of num
    cout << "Address of num: " << &num << endl;
    // Display the address stored in ptr
    cout << "Value of ptr (address): " << ptr << endl;
    // Display the value pointed to by ptr (dereference)
    cout << "Value pointed by ptr: " << *ptr << endl;
    return 0;
}
```

Output

```
Value of num: 42
Address of num: 0x61ff08
Value of ptr (address): 0x61ff08
Value pointed by ptr: 42
```

Task 2: Pointer Arithmetic:

Declare an integer array, use a pointer to traverse and display its elements using pointer arithmetic.

```
#include <iostream> // for input/output operations
using namespace std;
int main() {
```

```
// Declare an integer array with 5 elements
int numbers[] = {10, 20, 30, 40, 50};

// Declare pointer and initialize to point to first array element
int *ptr = numbers;

// Store the size of the array
int size = 5;

// Display message
cout << "Array elements using pointer arithmetic:" << endl;

// Loop through array using pointer arithmetic
for (int i = 0; i < size; i++) {

    // Access array element using pointer arithmetic (ptr + i)
    cout << "numbers[" << i << "] = " << *(ptr + i) << endl;

}

return 0;
}
```

Output

```
Array elements using pointer arithmetic:
numbers[0] = 10
numbers[1] = 20
numbers[2] = 30
numbers[3] = 40
numbers[4] = 50
```